# ЛЕКЦИЯ

**ГРАФЫ**

# ОСНОВНЫЕ ПОНЯТИЯ

**Граф** – пара G = <V, E>,

V – конечное непустое множество **вершин**,

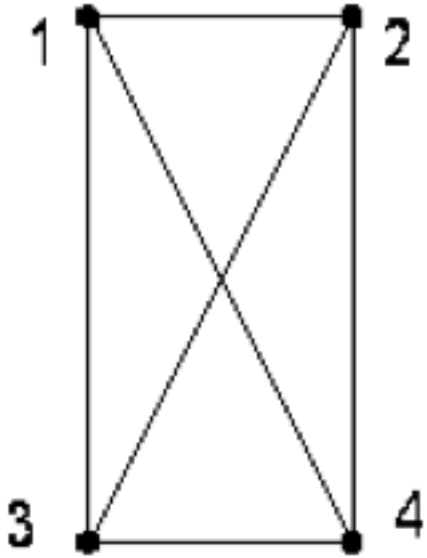
E – множество **ребер** (пар вершин).

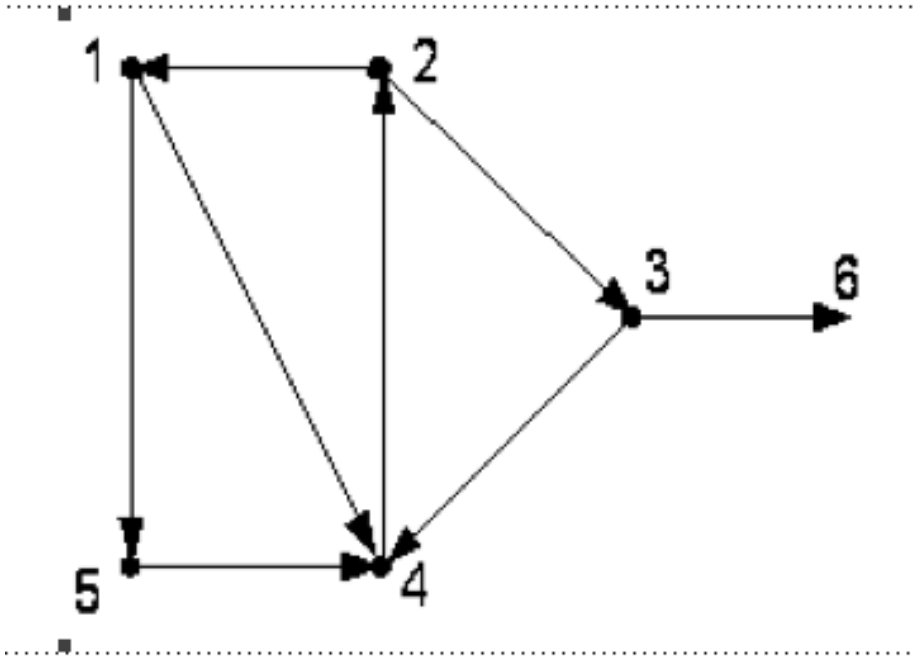
Если пары E не упорядочены – **граф неориентированный**

Если в пары E входят только различные вершины – **граф без петель**.

**Вершины** $V_1$ и $V_2$ называются **смежными**, если существует ребро E, соединяющее их. В таком случае говорят, **вершины инцидентны ребру** E или **ребро** E **инцидентно вершинам** $V_1$ и $V_2$.

**Ребра** называют **смежными**, если они имеют хотя бы одну общую вершину.



# ОСНОВНЫЕ ПОНЯТИЯ

**Путь в графе** – это последовательность ребер, в которой все ребра различные. Если при этом и все вершины различные (за исключением, может быть $V_0 = V_n$), то такой путь называется **простым**.

Последовательность ребер называется **замкнутой**, если $V_0 = V_n$.

Замкнутый простой путь в графе, содержащий, по крайней мере, одно ребро, называется **циклом**.

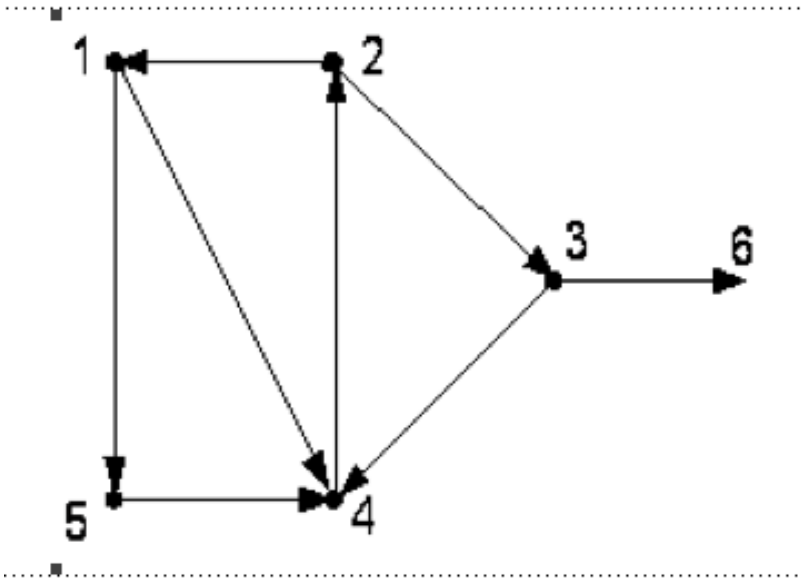
**Граф** называется **связным**, если для любой пары различных вершин существует соединяющий их путь.

# СПОСОБЫ ПРЕДСТАВЛЕНИЯ ГРАФА

- Матрица инцидентности
- Матрица смежности
- Список инцидентности

# МАТРИЦА ИНЦИДЕНТНОСТИ

Матрица инцидентности содержит n строк, соответствующих вершинам, и m столбцов, соответствующих ребрам.




Для ориентированного графа столбец, соответствующий ребру <x, y>, содержит:

1 в строке, соответствующей вершине y;

-1 – в строке, соответствующей вершине x;

0 – в остальных случаях.

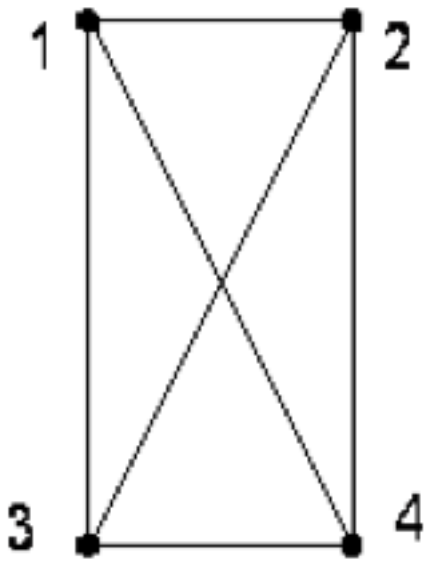
| | (1, 4) | (1, 5) | (2, 1) | (4, 2) | (2, 3) | (3, 4) | (3, 6) | (5, 4) |
|---|---|---|---|---|---|---|---|---|
| 1 | -1 | -1 | 1 | 0 | 0 | 0 | 0 | 0 |
| 2 | 0 | 0 | -1 | 1 | -1 | 0 | 0 | 0 |
| 3 | 0 | 0 | 0 | 0 | 1 | -1 | -1 | 0 |
| 4 | 1 | 0 | 0 | -1 | 0 | 1 | 0 | 1 |
| 5 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | -1 |
| 6 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |

# МАТРИЦА ИНЦИДЕНТНОСТИ

Для неориентированного графа столбец <x, y> содержит:
1 в строках, соответствующих вершинам x и y;
0 – в остальных строках.

Каждый столбец содержит ровно две единицы (или 1 и -1 для ориентированного графа), все столбцы различны.

Матрица инцидентности требует достаточно много памяти (n×m ячеек) и мало эффективна, поскольку часть ячеек заполнена нулями.



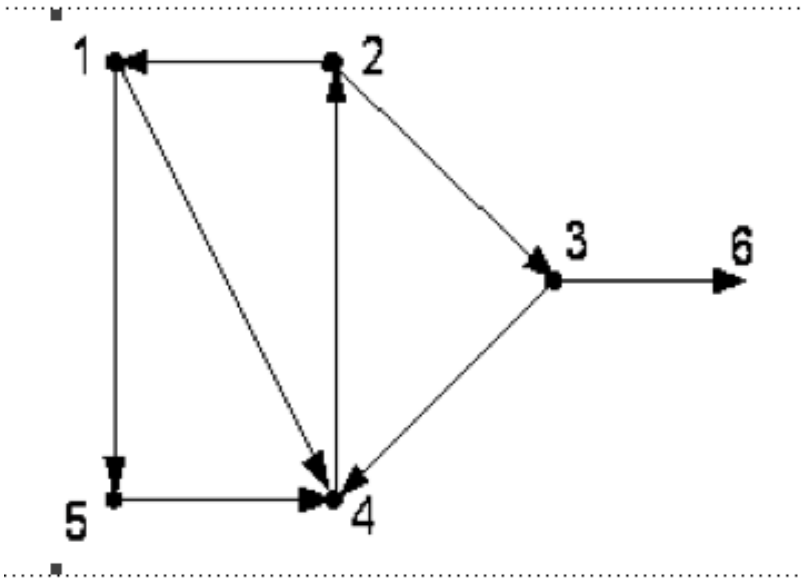

| | (1, 2) | (1, 3) | (1, 4) | (2, 3) | (2, 4) | (3, 4) |
|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 0 | 0 | 0 |
| 2 | 1 | 0 | 0 | 1 | 1 | 0 |
| 3 | 0 | 1 | 0 | 1 | 0 | 1 |
| 4 | 0 | 0 | 1 | 0 | 1 | 1 |

# МАТРИЦА СМЕЖНОСТИ

Матрица смежности содержит n×n элементов,

где n – количество вершин

Элемент матрицы равен 1, если есть ребро из вершины x в вершину y, и 0 в противном случае.



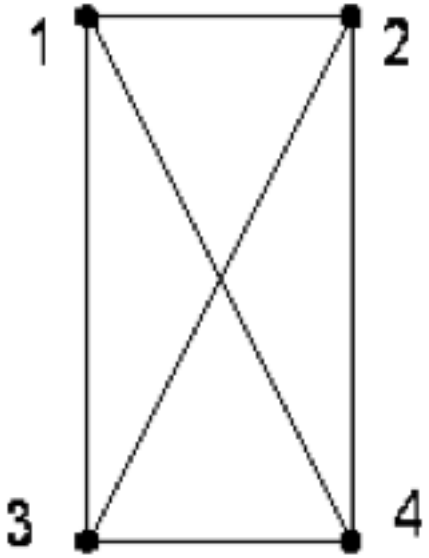

| | | | | | |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 1 | 0 |
| 1 | 0 | 1 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 1 |
| 0 | 1 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 |

# МАТРИЦА СМЕЖНОСТИ

У неориентированных графов матрица смежности всегда симметрична относительно главной диагонали.

Представление графа матрицей смежности за один шаг дает ответ на вопрос, есть ли путь из вершины x в вершину y.

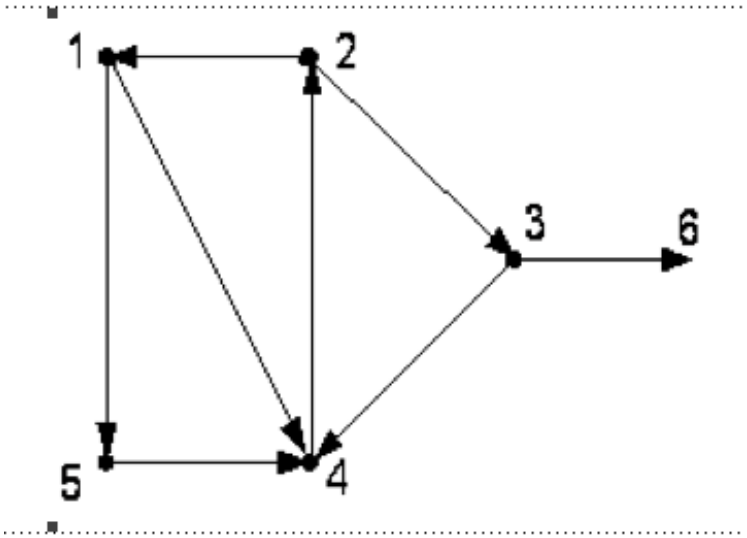
Матрица смежности требует n×n ячеек памяти.




| | | | |
|---|---|---|---|
| 0 | 1 | 1 | 1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 0 |

# СПИСОК ИНЦИДЕНТНОСТИ

**Список инцидентности** – это связанный список, где для каждой вершины x строится список вершин, смежных с ней.



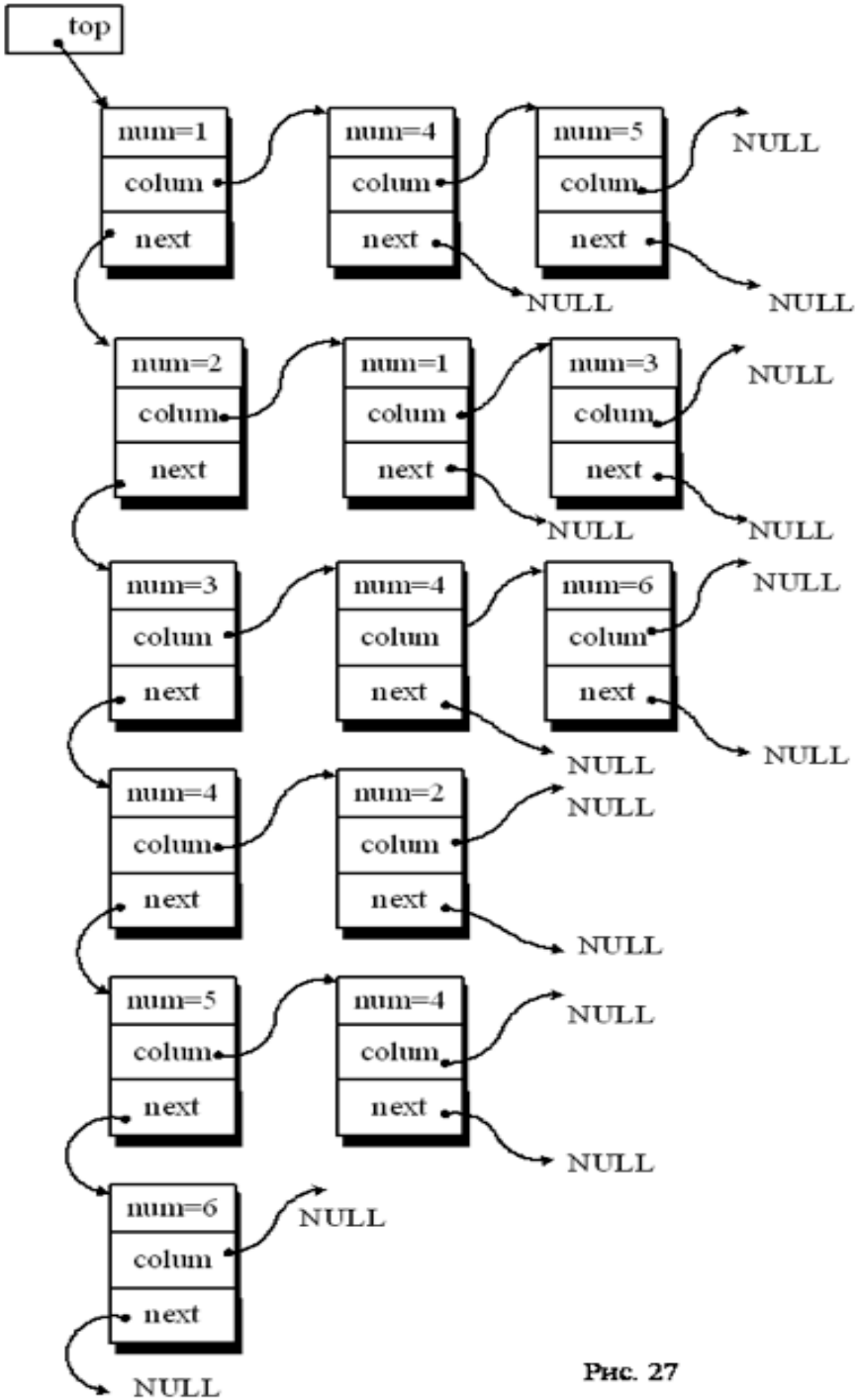




Рис. 27

Пример 1.

Представление графа в виде матрицы смежности и списка инцидентности. Список инцидентности строится по матрице смежности

Построение списков существующих путей из каждой вершины.

```
#include <conio.h>
#include <iostream>
#include <stdlib.h>
#include <time.h>
using namespace std;
// Тип узла списка
struct Row {
int Num; // Номер вершины
Row *Next; // Указатель на следующую вершину по порядку
Row *Colum; // Указатель на вершину, в которую существует путь
};
// Формирование матрицы смежности
void GetMas(int **A, int N)
{
int i, j, x;
srand((unsigned)time(NULL));
// Заполняем матрицу
for (i = 0; i < N; i++)
  {for(j = 0; j < N; j++)
       {x = rand()%100;
        if (x > 50)
           A[i][j] = 1;
       else  A[i][j] = 0;
        }
   }
 for (i = 0; i < N; i++) // Выставляем нули на главной диагонали
   A[i][i] = 0; }
// Вывод матрицы смежности
void OutMas(int **A, int N)
{
int i, j;
cout << '|' << ' ';
for (i = 0; i < N; i++)
  { cout < i < ' '; }
cout << '\n';
for (i = 0; i < N; i++)
  { cout << i << ' ';
    for (j = 0; j < N; j++)
      { cout << A[i][j] << ' '; }
   cout << '\n';   }
}
```

```
// Преобразование матрицы смежности в список инцидентности
Row *Convert(int **A, int N, Row *Top)
{
int i, j;
Row *P, *T, *C;
// Построение очереди из вершин графа по порядку нумерации
// Построение первого узла очереди
P = new Row;
P->Num = 0;
P->Next = NULL;
P->Colum = NULL;
Top = P; // Сохраняем вершину очереди
// Построение остальных узлов очереди
for (i = 1; i < N; i++)
{ T = new Row;

 T->Num = i;
 T->Next = NULL;
 T->Colum = NULL; // Сюда будем присоединять вершины,
  // в которые существует путь
P->Next = T;
P = T;
}
cout << '\n' << "Распечатаем очередь вершин" << '\n';
P = Top;
while (P != NULL)
{cout << '\n' << P->Num;
P = P->Next;
}
// Построение списков существующих путей из каждой вершины
T = Top; P = Top;
for (i = 0; i < N; i++)
{
  for (j = 0; j < N; j++)
   {  if (A[i][j] == 1) // Проверка существования пути
       { C = new Row; // Добавляем новый узел по указателю
      C->Num = j;//Заносим туда номер вершины, куда есть путь
         C->Next = NULL;
         C->Colum = NULL;
         P->Colum = C;
         P = C;
        }
    }
```

```
  T = T->Next; // Переходим к следующей вершине
  P = T;
 }
 return Top;
}
// Вывод списка инцидентности
void OutSpis(Row *Top)
{
 Row *P, *T;
 P = Top;
 T = P->Colum;
 // Пока не пройдены все вершины
 while (P != NULL)
   {cout << P->Num << ' ';
// Пока не пройдены все вершины, в которые существует
// путь из данной вершины P, выводим их номера
    while (T != NULL)
     {cout << T->Num << ' ';
      T = T->Colum;
     }
     P = P->Next;
     T = P->Colum;
     cout << '\n';
   }
}
void main()
{ Row *Top;
  int **A;
  system("cls”);
  int N;
  cout << "\nВведите количество вершин графа";
  cin >> N;
  A = new int *[N];
  for (int i = 0; i < N; i++)
    {A[i]=new int[N];   }
  GetMas(A, N);
  OutMas(A, N);
  Top = Convert(A, N, Top);
  cout << '\n' << Top->Num << '\n';
  cout << "Список инцидентности\n";
   OutSpis(Top);
   getch();
}
```

Пример 2.

Находятся вершины, недостижимые из данной вершины. Строится список инцидентности по матрице смежности. Если вершина находится в списке доступных вершин из данной вершины, то вершины, смежные с первой вершиной, тоже доступны.

```
#include <conio.h>
#include <iostream>
#include <time.h>
using namespace std;

// Тип узла списка
struct Row {
int Num;
Row *Next;
Row *Colum; };

// Задание матрицы смежности
void GetMas(int **A, int N)
{   int i, j, x;
srand((unsigned)time(NULL));
for (i = 0; i < N; i++)
  { for (j = 0; j < N; j++)
      {x = rand()%100;
      if (x > 50) A[i][j] = 1;
      else A[i][j] = 0;     }   }
for (i = 0; i < N; i++)
  A[i][i] = 0;
}

// Вывод матрицы смежности
void OutMas(int **A, int N)
{   int i, j;
cout << '|' << ' ';
for (i = 0; i < N; i++)
 {  cout << i << ' ';   }
cout << '\n';
for (i = 0; i < N; i++)
 { cout << i << ' ';
   for (j = 0; j < N; j++)
     {  cout << A[i][j] << ' ';     }
  cout << '\n';
  }
}
```

```
// Преобразование матрицы смежности в список инцидентности
Row *Convert(int **A, int N, Row *Top)
{   int i, j;
Row *P, *T, *C;
P = new Row;
P->Num = 0;
P->Next = NULL;
P->Colum = NULL;
Top = P;
for (i = 1; i < N; i++)
   {   T = new Row;
       T->Num = i;
      T->Next = NULL;
      T->Colum = NULL;
      P->Next = T;
      P = T;
     }
    T = Top;
    P = Top;
    for (i = 0; i < N; i++)
      {for (j = 0; j < N; j++)
         { if (A[i][j] == 1)
            { C = new Row;
              C->Num = j;
              C->Next = NULL;
              C->Colum = NULL;
              P->Colum = C;
              P = C;
            }
         }
        T = T->Next;
        P = T;
       }
    return Top; }
```

```
// Вывод списка инцидентности
void OutSpis(Row *Top)
{   Row *P, *T;
P = Top;
T = P->Colum;
while (P != NULL)
   { cout << P->Num << ' ';
     while (T != NULL)
       {  cout << T->Num << ' ';
          T = T->Colum;
       }
     P = P->Next;
     T = P->Colum;
     cout << '\n';    } }

// Есть ли j в X
  int in(int j, int *X, int n)
   { for (int i = 0; i < n; i++)
     if (j == X[i]) return 1;
    return 0;
   }
```

```
// Чтобы найти недостижимые, находим достижимые
void Find(Row *Top, int n1, int n)
{   int i, k;
int *Check = new int[n]; //Массив, определяющий проверена ли вершина
int *R = new int[n]; // Массив, содержащий номера вершин, доступных из данной
Row *P, *T;
for (i = 0; i < n; i++)
{ Check[i] = 0;
 R[i] = -1;   }
k = 0;
R[k++] = n1;
for (i = 1; i < n; i++) // Перебираем все вершины n раз
 { P = Top; // Проходим по всем вершинам
 while (P != NULL)
   {if (in(P->Num, R, n) && Check[i] == 0)
     { // Если вершина попадает в список проходимых вершин и не проверялась
    T = P->Colum;
    while (T != NULL)
      { if (in(T->Num, R, n) == 0)
        {// Заносим все смежные вершины
        // в список достижимых
        R[k++] = T->Num;
        Check[P->Num] = 1; // Отмечаем как проверенную
      }
      T = T->Colum;
       }
     }
 P = P->Next;
  }
}
cout << "\n Список недостижимых вершин \n";
int fl = 0;
for (i = 0; i < n; i++)
  {if (in(i, R, n) == 0)
      { cout << i << '\t';
       fl = 1;
      }
   }
if (fl == 0)
   cout << "\n Все вершины достижимы";
}
```

```
void main()
{   Row *Top;
int **A;
system("cls”);
int N;
cout << "\n Введите количество вершин графа";
cin >> N;
A = new int *[N];
for (int i = 0; i < N; i++)
   { A[i] = new int[N]; }
 GetMas(A, N);
OutMas(A, N);
Top = Convert(A, N, Top);
cout << "Список инцидентности";
OutSpis(Top);
cout << "\nВведите номер вершины\n";
int n1;
cin >> n1;
Find(Top, n1, N);
getch();
}
```

# АЛГОРИТМ DFS

Часто возникает задача **определения пути между двумя вершинами графа**. Для решения этой задачи подойдет **алгоритм поиска в глубину – алгоритм DFS** (*depth first search*).

Большинство алгоритмов на графах в своей основе имеют систематический перебор вершин графа. Алгоритм DFS рекурсивный и работает следующим образом: проходим первое встретившееся ребро из начальной вершины Start в W. Если вершина W не совпадает с вершиной Finish, и не была просмотрена, то вершина W становится начальной, алгоритм рекурсивно обращается сам к себе и далее поиск продолжается от вершины W. Если не просмотренных вершин, смежных с W не обнаружено, алгоритм возвращается в вершину, из которой он опустился в W, и проверяет другие смежные ребра. Иными словами, поиск в глубину из вершины Start основан на поиске в глубину из всех новых вершин, смежных с Start. При поиске в глубину, чем позднее посещается вершина, тем раньше она используется. Однако **определенный таким образом путь не является кратчайшим**, и это главный недостаток поиска в глубину.

Пример 3.

Определение пути между двумя вершинами графа. Граф представлен в виде матрицы смежности map[N][N], в массив road[N], записываются номера, пройденных вершин, т.е. путь.

Массив incl[10] определяет, включена ли вершина в маршрут, чтобы какую-либо вершину не включить в маршрут дважды.

```
#include <iostream>
#include <conio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>
using namespace std;
const int N = 10;
int map[N][N],
    incl[10], //Включена ли точка в маршрут
    road[10], //Номера пройденных вершин
    f1 = 0;

// Задание матрицы смежности


void GetMas(int n)
{int i,j,x;
srand((unsigned)time(NULL));
for(i=0;i<n;i++)
   for(j=0;j<n;j++)
   {x=rand()%100;
    if(x>50)
      map[i][j]=1;
    else   map[i][j]=0;
   }
for(i=0;i<n;i++)
   map[i][i]=0;

}

//Вывод матрицы смежности на экран

void OutMas(int n)
{int i,j;
cout<<'|'<<' ';
for(i=0;i<n;i++)
   cout<<i<<' ';
cout<<'\n';
for(i=0;i<n;i++)
   {cout<<i<<' ';
   for(j=0;j<n;j++)
      cout<<map[i][j]<<' ';
   cout<<'\n';
   }
}
```

```
//Поиск пути из вершины start в вершину finish

void Step (int s, int f, int p, int n)
//p - номер текущей точки маршрута
{ int c,i;
if(s==f) //Тривиальное решение: вершины s и f совпали
  {//Вывод пути на экран
   cout<<"Way";
   f1=1;
   for(i=0;i<=p-1;i++)
      cout<<road[i]<<' ';
   cout<<'\n';
   }
else
   {for(c=0;c<n;c++) //Перебираем все вершины
       {//Если есть соединение и вершина не включена в маршрут
        if(map[s][c]==1 && incl[c]==0)
          {road[p]=c; //Включаем вершину в путь
           incl[c]=1; //Отмечаем вершину как пройденную
           Step(c,f,p+1,n);//Ищем, есть ли путь из c в f
           if(f1==0)
              {//Если путь не найден, то на выходе из рекурсии
               //исключаем последнюю вершину из маршрута
               incl[c]=0;
               road[p]=-10;
              }
            else break;
          }
        }
    }
}

void main ()
{int n,i,j,start,finish;
setlocale(LC_ALL,"");
cout<<"Введите количество вершин:";
cin>>n;
GetMas(n);
OutMas(n);
cout<<"Введите номера начальной и конечной вершин:";
cin>>start>>finish;
road[0]=start;
incl[start]=1;
Step(start,finish,1,n);
if (f1==0) cout<<"Пути нет";
getch();
}
```

# АЛГОРИТМ BFS

Для нахождения **кратчайшего пути в графе** используется метод, основанный на **поиске в ширину** – **алгоритм BFS** (*breadth first search, BFS*).

Алгоритм BFS рекурсивный и работает следующим образом: из начальной вершины first проходим во все смежные вершины, если среди них нет искомой вершины last, то поиск осуществляется среди вершин, смежных с ними, в той последовательности, в которой они просматривались, и т.д.

В алгоритме поиска в ширину получается, чем раньше посещается вершина, тем раньше она используется. Использование вершины производится просмотром сразу всех новых соседей этой вершины.

# ПРИМЕР НАХОЖДЕНИЯ КРАТЧАЙШЕГО ПУТИ НА ГРАФЕ

Пример 4.

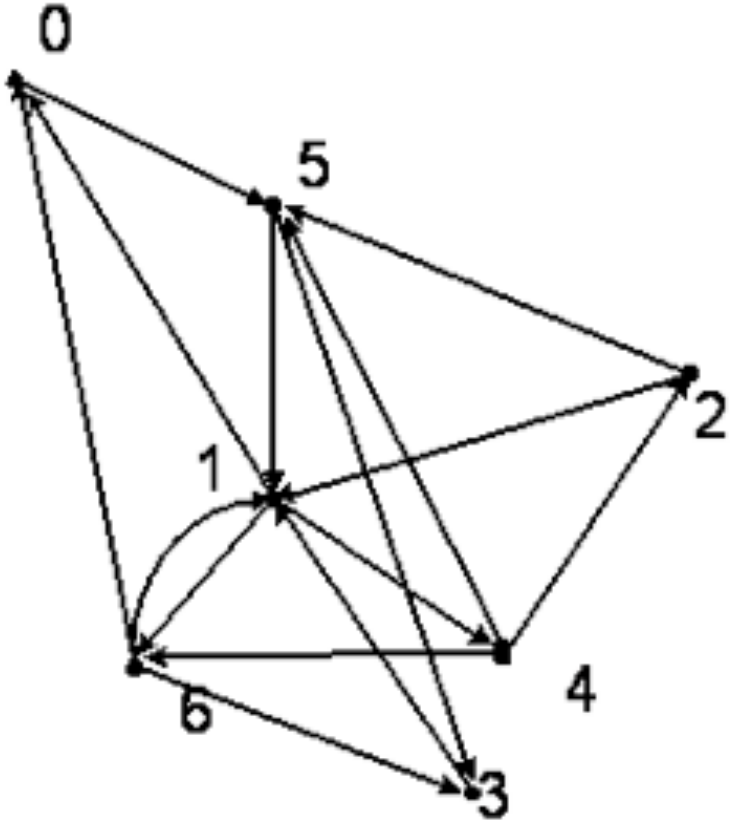
Определение кратчайшего пути между двумя вершинами графа.

Граф представлен в виде списка инцидентности sp[max], организованного так, что в массиве sp элемент sp[i] хранит указатель на список вершин, смежных с вершиной i. В массив labels[max] записываются номера пройденных вершин, чтобы какую-либо вершину не включить очередь поиска дважды.

Кратчайший путь выводится путем передвижения по родительским ссылкам от конечной вершины пути к начальной.

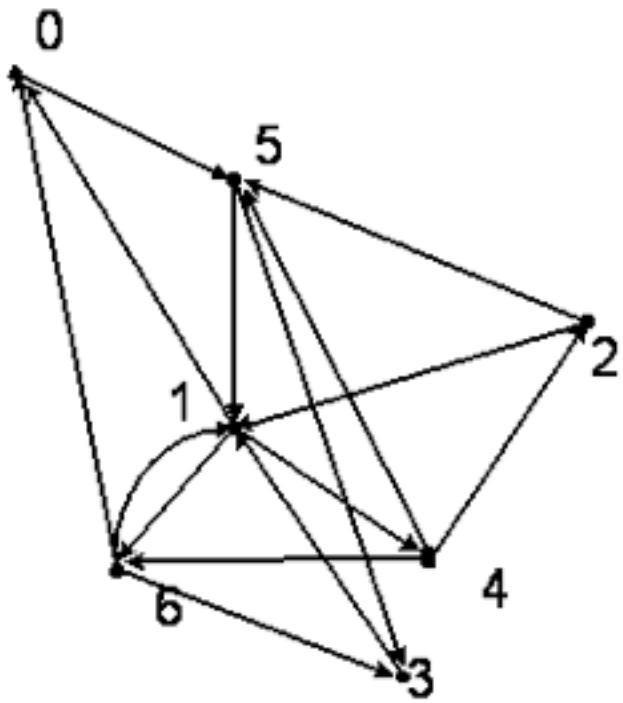
# ПРИМЕР НАХОЖДЕНИЯ КРАТЧАЙШЕГО ПУТИ НА ГРАФЕ

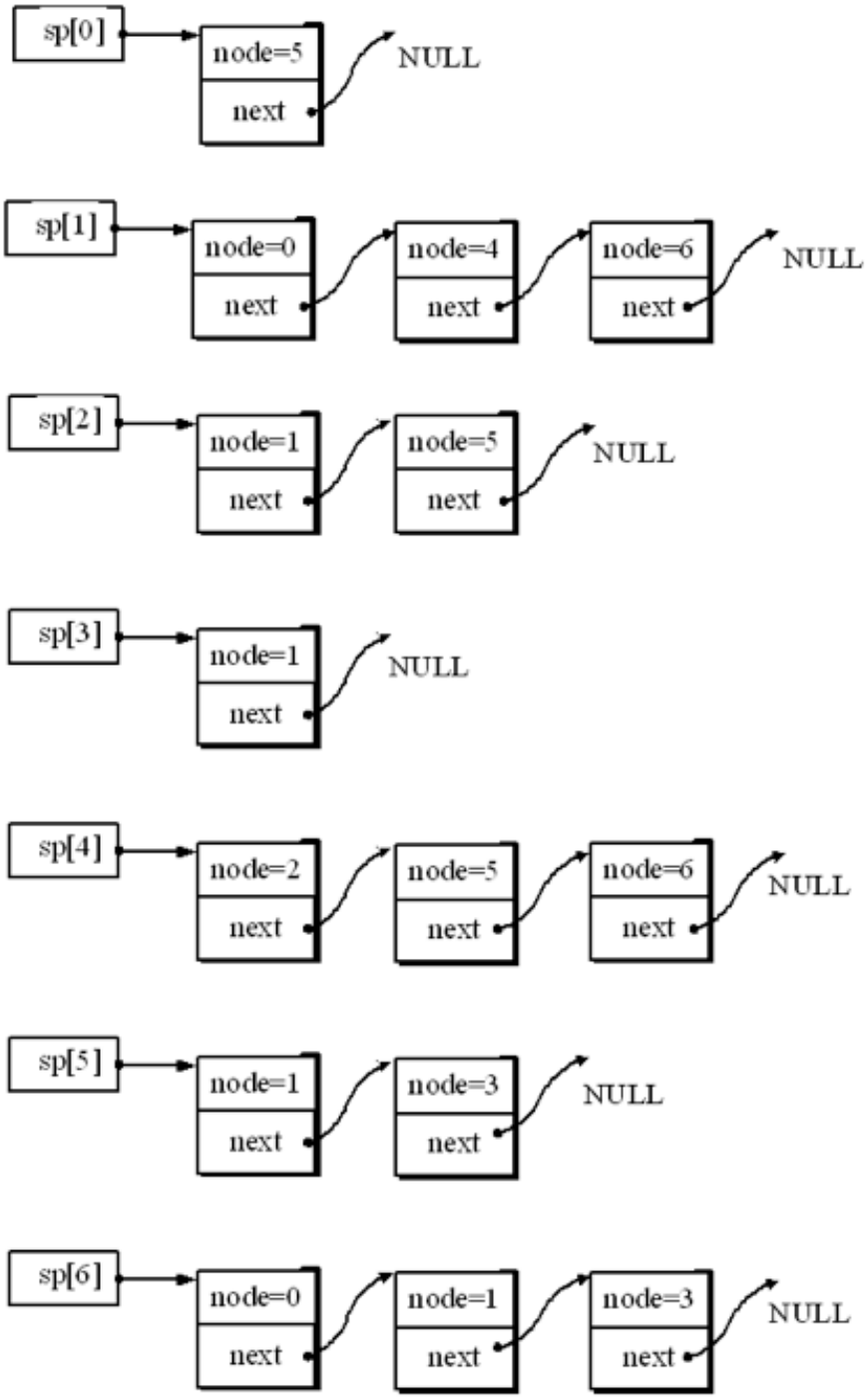
Для графа, изображенного на рисунке, при вводе в качестве начальной вершины 5, а конечной – 2, будет выведен следующий путь 2 4 1 5.



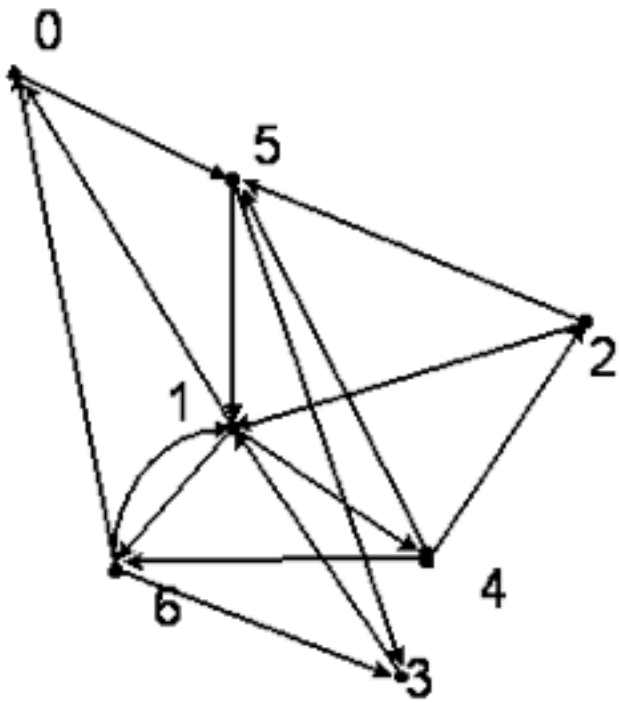
Список инцидентности:

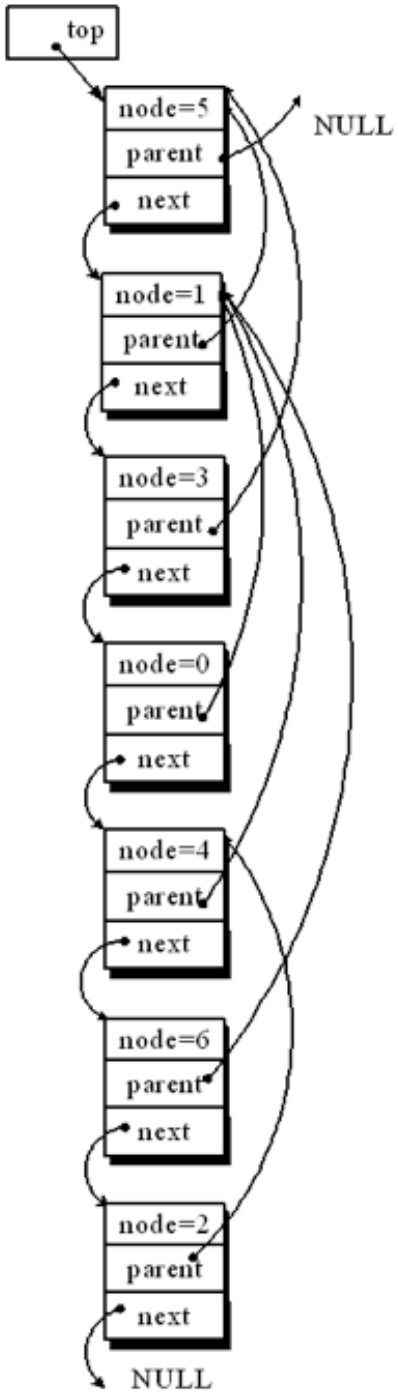
sp[i] хранит указатель на список вершин, смежных с вершиной i.



Очередь поиска кратчайшего пути из вершины 5 в вершину 2



```
// Поиск кратчайшего пути в ширину рекурсивно
#include <iostream>
#include <conio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>
#define max 7
using namespace std;

//const int max=7;
// Тип, используемый в списке инцидентности
struct rec1{
int node;
rec1 *next;};

// Тип, используемый в очереди поиска
struct rec2{
int node;
rec2 *next; // Указатель на следующий узел в очереди поиска
rec2 *parent; // Указатель на родительский узел
};

rec1 *sp[max]; // Массив указателей на список инцидентности
int first, last, k=0;
rec2 *top, *iter;
int labels[max]; // Массив с пройденными вершинами
// Построение списка инцидентности

void randgraf()
{  int i, j, x, first;
rec1 *ptr;
for (i = 0; i < max; i++)
   {first = 1;
    for (j = 0; j < max; j++)
     {
     //Вероятность появления связи
     x=rand()%10+1;
      if (x > 5 && i != j)
         {if (first)
          {
// Построение первого узла списка для i вершины
          first = 0;
          sp[i] = new rec1;
          ptr = sp[i];
          ptr->next = NULL;
         ptr->node = j;  }
            else
// Построение остальных узлов списка для i вершины
           {ptr->next = new rec1;
              ptr = ptr->next;
              ptr->next = NULL;
              ptr->node = j;
            } } } }}

// Вывод списка инцидентности
void outgraf()
{  rec1 *ptr;
int i;
for (i = 0; i < max; i++)
{ptr = sp[i];
cout << i << ' '; // Вывод номера узла
while (ptr != NULL)
 {cout << ptr->node << ' ';
  ptr = ptr->next; }
cout << '\n';  }}

// Проверка, принадлежит ли j массиву X
int in(int j, int *X, int n)
{  for (int i = 0; i < n; i++)
     {if (j == X[i]) return 1; }
return 0;}
```

```
// Вывод кратчайшего пути
void successful(rec2* bufer)
{// Если конечный узел найден, то возвращаясь от него по
// родительским ссылкам, выводим найденный кратчайший путь
cout << "Путь";
while (bufer != NULL)
 {cout << bufer->node << " ";
   bufer = bufer->parent;  }
getch();}

// Построение очереди поиска
void Addusel(rec2 *buf)
// Добавляет узлы, смежные с данным в конец очереди поиска
{ rec1 *bufer;
bufer = sp[buf->node];
while ((bufer!= NULL) && (iter->node!=last))
  { if (!in(bufer->node, labels, max))
      {iter->next = new rec2;
       iter = iter->next; // На конец очереди указывает iter
       iter->next = NULL;
       iter->node = bufer->node;
    // Помечаем узел как пройденный
       labels[k++] = iter->node;
       iter->parent = buf;
      }
    else
      bufer = bufer->next;
   }
}

// Поиск кратчайшего пути в ширину рекурсивно
rec2* search(rec2 *bufiter)
{

if (iter->node == last)
     return iter; // Узел найден - тривиальное решение
else
    if (bufiter != NULL)
      {// Присоединяем узлы смежные с данным
      Addusel(bufiter);
      // Исследуем следующий узел в очереди поиска
     search(bufiter->next);  }
 // Если очередь исследована, а узел не найден
     else  return NULL;  }

void main()
{  rec2 *resultat;
  setlocale(LC_ALL,"");
 srand((unsigned)time(NULL));

for (int i = 0; i < max; i++)
    labels[i] = -1;
for (int i = 0; i < max; i++)
   sp[i] = NULL;
randgraf();
outgraf();
cout << "Введите первый и последний узлы";
cin >> first >> last;
top = new rec2;
top->parent = NULL;
top->next = NULL;
top->node = first;
labels[k++] = top->node;
 iter = top;
resultat =search(iter);
if (iter != NULL)
   successful(iter);
else cout<<"Путь не найден";
getch();}
```